

Machine Learning for Climate Risk Prediction

Flood Risk Prediction using Explainable AI

Objectives

- Predict flood-prone areas using environmental indicators
- Compare machine learning models
- Explain predictions using SHAP explainability
- Discuss ethical implications of AI-driven climate-risk systems

```
In [1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

from xgboost import XGBClassifier

import shap

import warnings
warnings.filterwarnings('ignore')
```

```
In [4]: df = pd.read_csv("data/train.csv")
```

```
In [5]: df.head()
```

Out[5]:

	id	MonsoonIntensity	TopographyDrainage	RiverManagement	Deforestation	Urban
--	----	------------------	--------------------	-----------------	---------------	-------

0	0	5	8	5	8
1	1	6	7	4	4
2	2	6	5	6	7
3	3	3	4	6	5
4	4	5	3	2	6

5 rows × 22 columns



In [6]: `df.columns`

Out[6]: Index(['id', 'MonsoonIntensity', 'TopographyDrainage', 'RiverManagement', 'Deforestation', 'Urbanization', 'ClimateChange', 'DamsQuality', 'Siltation', 'AgriculturalPractices', 'Encroachments', 'IneffectiveDisasterPreparedness', 'DrainageSystems', 'CoastalVulnerability', 'Landslides', 'Watersheds', 'DeterioratingInfrastructure', 'PopulationScore', 'WetlandLoss', 'InadequatePlanning', 'PoliticalFactors', 'FloodProbability'], dtype='object')

In [7]: `df.shape`

Out[7]: (1117957, 22)

In [8]: `df = df.drop("id", axis=1)`

In [9]: `X = df.drop("FloodProbability", axis=1)`
`y = df["FloodProbability"]`

In [10]: `from sklearn.model_selection import train_test_split`
`X_train, X_test, y_train, y_test = train_test_split(`
 `X,`
 `y,`
 `test_size=0.2,`
 `random_state=42`
`)`

In [11]: `from sklearn.preprocessing import StandardScaler`
`scaler = StandardScaler()`
`X_train_scaled = scaler.fit_transform(X_train)`
`X_test_scaled = scaler.transform(X_test)`

In [12]: `from sklearn.linear_model import LinearRegression`
`lr_model = LinearRegression()`

```
lr_model.fit(X_train_scaled, y_train)

lr_pred = lr_model.predict(X_test_scaled)
```

```
In [13]: from sklearn.metrics import (
          mean_absolute_error,
          mean_squared_error,
          r2_score
        )

mae = mean_absolute_error(y_test, lr_pred)

mse = mean_squared_error(y_test, lr_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, lr_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

MAE: 0.015792471363760326
RMSE: 0.02008000465862889
R2 Score: 0.8448773362840329

```
In [14]: from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)
```

```
In [15]: rf_mae = mean_absolute_error(y_test, rf_pred)

rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))

rf_r2 = r2_score(y_test, rf_pred)

print("Random Forest Results")
print("MAE:", rf_mae)
print("RMSE:", rf_rmse)
print("R2 Score:", rf_r2)
```

Random Forest Results
MAE: 0.02447473254857062
RMSE: 0.029940880090698195
R2 Score: 0.655113051951046

```
In [16]: from xgboost import XGBRegressor

xgb_model = XGBRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=6,
```

```

    random_state=42,
    n_jobs=-1
)

xgb_model.fit(X_train, y_train)

xgb_pred = xgb_model.predict(X_test)

```

```

In [17]: xgb_mae = mean_absolute_error(y_test, xgb_pred)

xgb_rmse = np.sqrt(mean_squared_error(y_test, xgb_pred))

xgb_r2 = r2_score(y_test, xgb_pred)

print("XGBoost Results")
print("MAE:", xgb_mae)
print("RMSE:", xgb_rmse)
print("R2 Score:", xgb_r2)

```

XGBoost Results
MAE: 0.020486236550541676
RMSE: 0.024923343449959626
R2 Score: 0.7610207247802385

```

In [18]: results = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Random Forest",
        "XGBoost"
    ],
    "MAE": [
        mae,
        rf_mae,
        xgb_mae
    ],
    "RMSE": [
        rmse,
        rf_rmse,
        xgb_rmse
    ],
    "R2 Score": [
        r2,
        rf_r2,
        xgb_r2
    ]
})

```

results

```

Out[18]:

```

	Model	MAE	RMSE	R2 Score
0	Linear Regression	0.015792	0.020080	0.844877
1	Random Forest	0.024475	0.029941	0.655113
2	XGBoost	0.020486	0.024923	0.761021

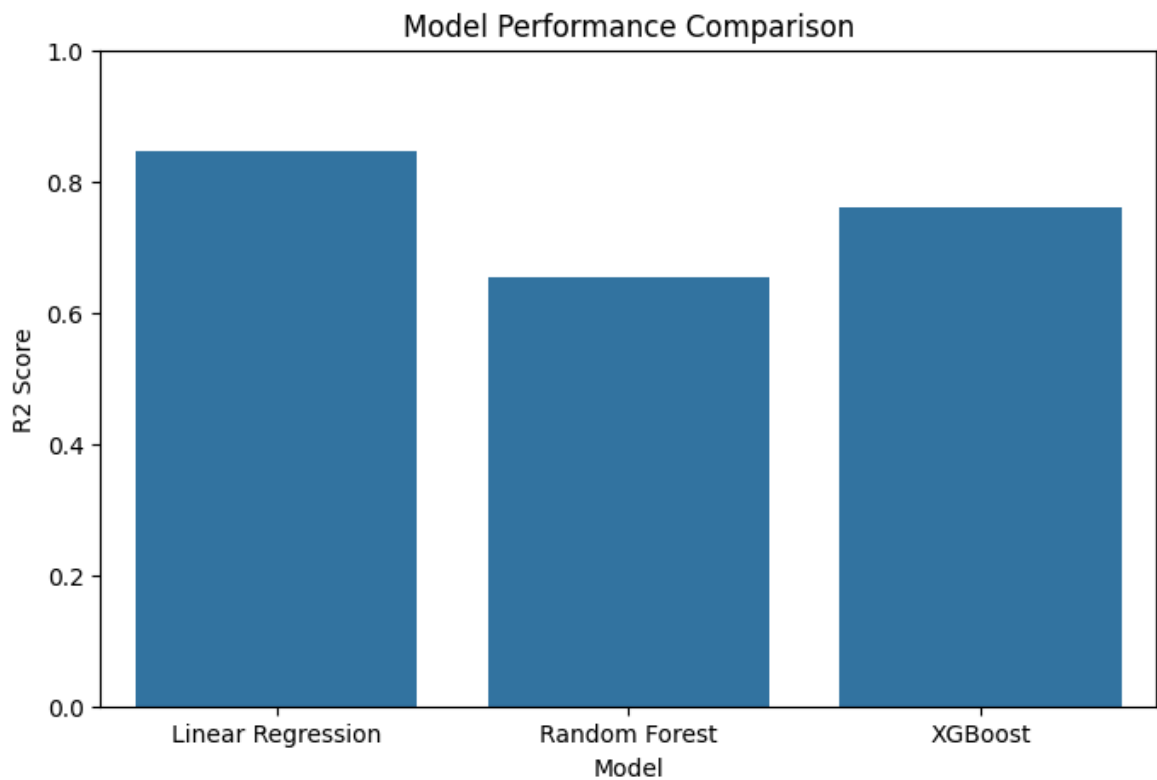
```

In [19]: plt.figure(figsize=(8,5))

sns.barplot(

```

```
x="Model",  
y="R2 Score",  
data=results  
)  
  
plt.title("Model Performance Comparison")  
  
plt.ylim(0,1)  
  
plt.show()
```



```
In [20]: importance = pd.DataFrame({  
        "Feature": X.columns,  
        "Importance": xgb_model.feature_importances_  
    })  
  
importance = importance.sort_values(  
    by="Importance",  
    ascending=False  
)  
  
importance.head(10)
```

Out[20]:

	Feature	Importance
15	DeterioratingInfrastructure	0.057434
0	MonsoonIntensity	0.056786
6	DamsQuality	0.055753
7	Siltation	0.054625
2	RiverManagement	0.054194
1	TopographyDrainage	0.054119
16	PopulationScore	0.053403
5	ClimateChange	0.053219
3	Deforestation	0.051044
17	WetlandLoss	0.050773

```

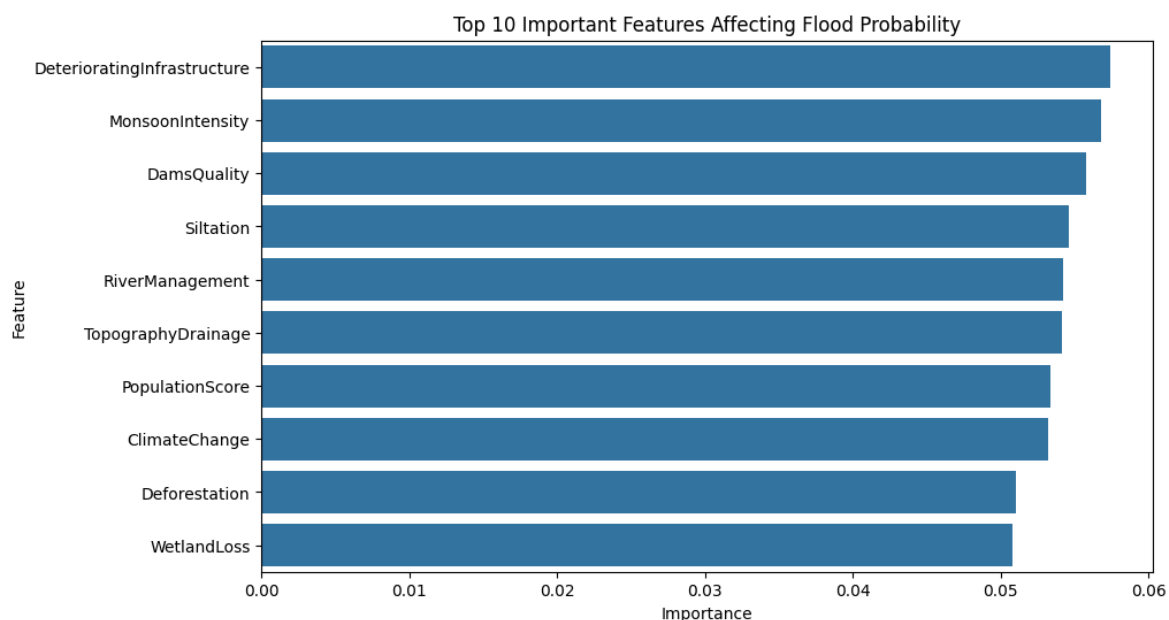
In [21]: plt.figure(figsize=(10,6))

sns.barplot(
    x="Importance",
    y="Feature",
    data=importance.head(10)
)

plt.title("Top 10 Important Features Affecting Flood Probability")

plt.show()

```



```

In [22]: explainer = shap.Explainer(xgb_model)

shap_values = explainer(X_test[:1000])

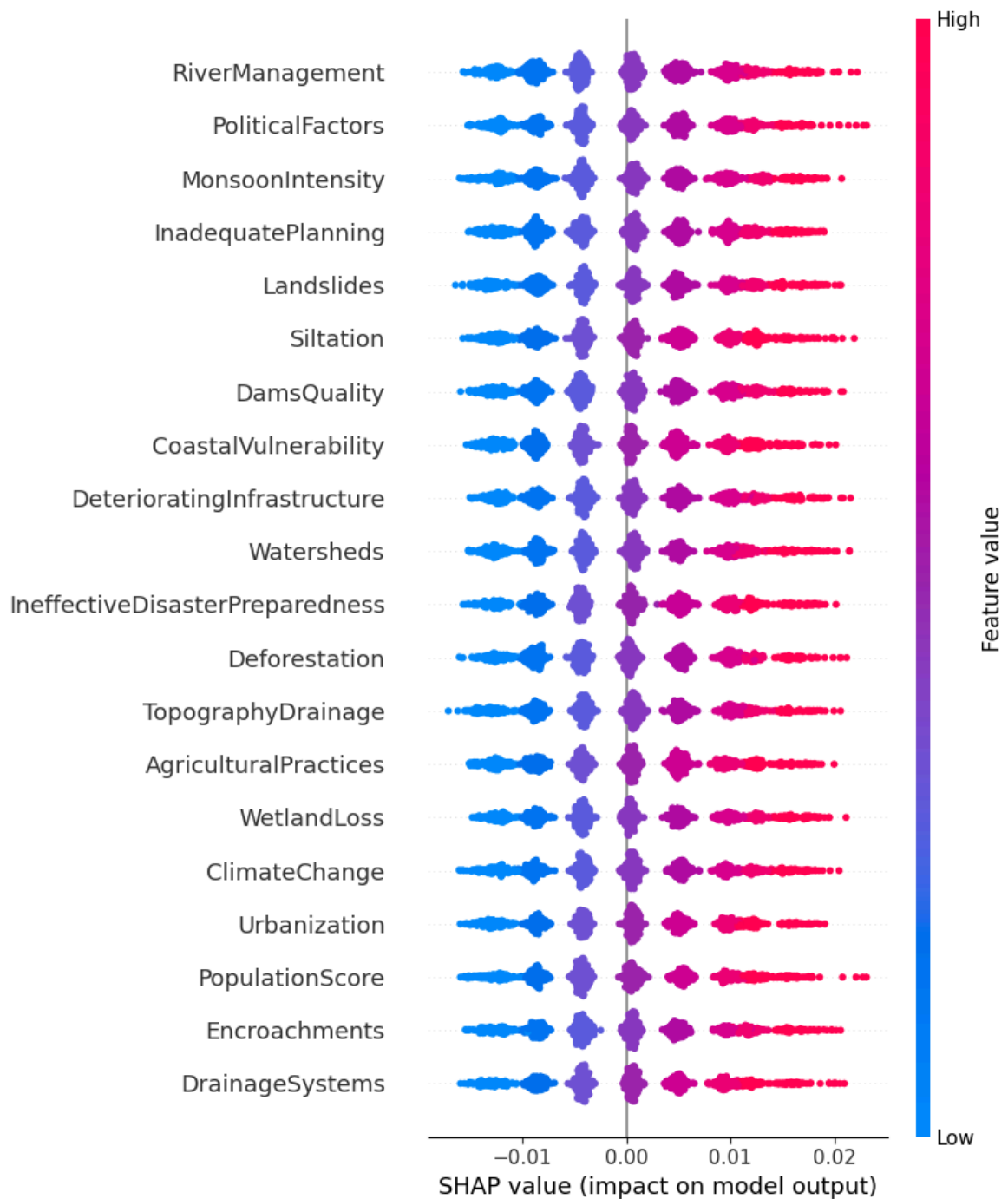
```

```

In [23]: shap.summary_plot(
    shap_values,

```

```
X_test[:1000]
)
```



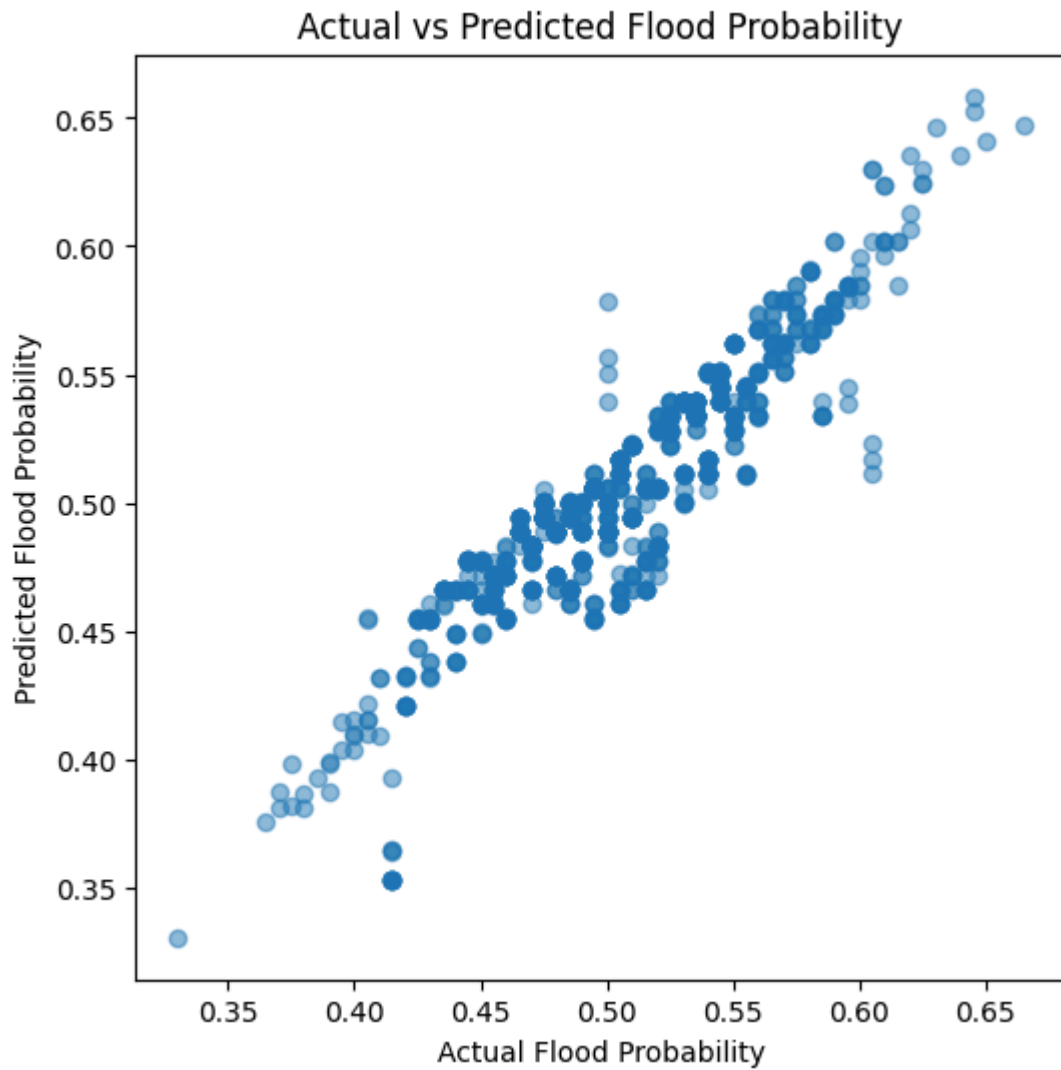
```
In [25]: plt.figure(figsize=(6,6))

plt.scatter(
    y_test[:1000],
    lr_pred[:1000],
    alpha=0.5
)

plt.xlabel("Actual Flood Probability")
plt.ylabel("Predicted Flood Probability")

plt.title("Actual vs Predicted Flood Probability")

plt.show()
```



```
In [26]: residuals = y_test - lr_pred

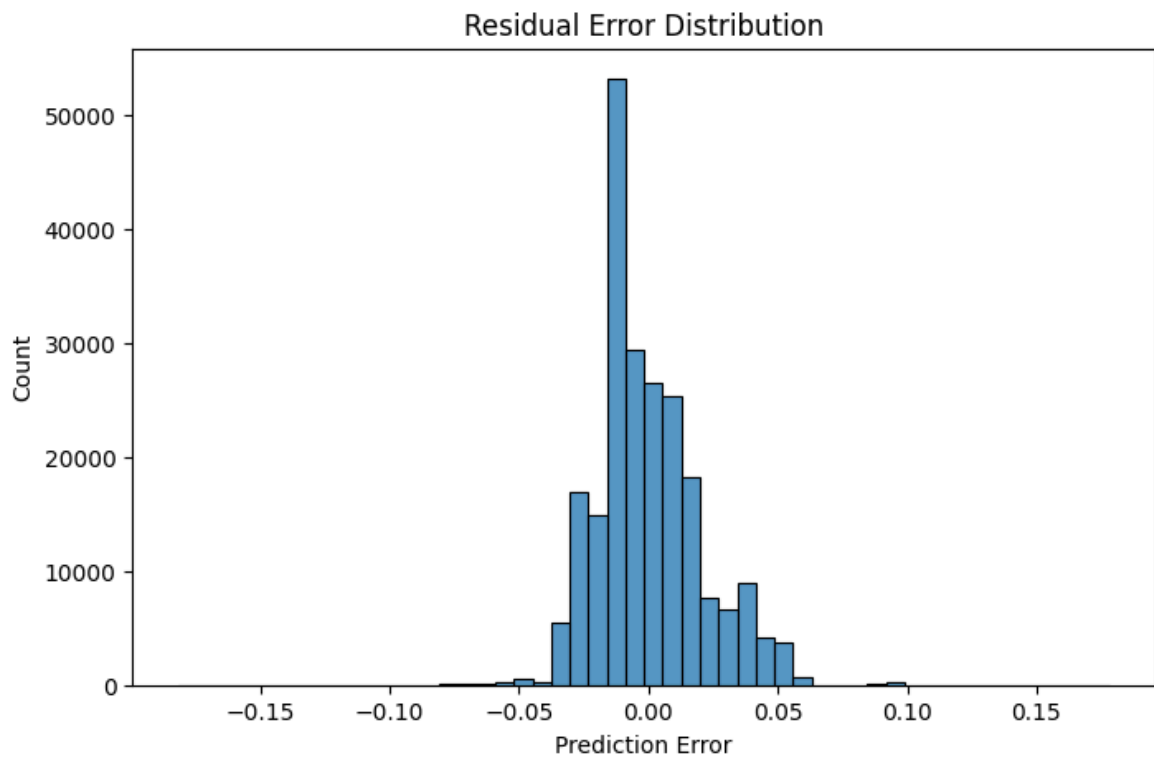
plt.figure(figsize=(8,5))

sns.histplot(
    residuals,
    bins=50
)

plt.title("Residual Error Distribution")

plt.xlabel("Prediction Error")

plt.show()
```



Research Questions

RQ1

Which machine learning model provides the best predictive performance for flood probability estimation?

RQ2

Which environmental factors contribute most significantly to flood probability prediction?

RQ3

How can explainable AI improve transparency in climate-risk prediction systems?

RQ4

What are the ethical implications of AI-driven flood prediction systems?

Results and Discussion

Linear Regression achieved the strongest predictive performance with an R^2 score of approximately 0.845, outperforming Random Forest and XGBoost models.

The results suggest that the relationships between environmental indicators and flood probability are predominantly linear in nature.

Feature importance analysis and SHAP explainability identified monsoon intensity, urbanization, drainage systems, and climate-related indicators as major contributors to flood probability prediction.

The findings demonstrate that simpler interpretable models may outperform more complex ensemble methods when the environmental relationships within the dataset exhibit strong linear characteristics.

Responsible AI Considerations

AI-driven climate-risk prediction systems introduce important ethical and societal considerations.

False negative predictions may result in inadequate disaster preparedness and delayed emergency response. False positive predictions may trigger unnecessary evacuations and economic disruption.

Bias within environmental datasets may reduce model reliability across underrepresented geographic regions.

Explainable AI techniques such as SHAP improve transparency and increase trustworthiness in environmental decision-support systems.

```
In [27]: import joblib  
  
         joblib.dump(lr_model, "flood_prediction_model.pkl")
```

```
Out[27]: ['flood_prediction_model.pkl']
```

```
In [ ]:
```